SQL SERVER | DATA ANALYTICS PROJECT

Data Cleaning in SQL

From 1,004 messy rows to a clean, analysis-ready table in 7 steps

Duplicates · Nulls · Negatives · Dates · Emails · Datatypes

By Amaan

Swipe > step-by-step walkthrough with real code

The Dataset & The Problems

E-commerce sales data | 1,004 rows | 13 columns

Before cleaning anything, I profiled the data. This is what I found in the raw file:

Problem	Found in raw data
Duplicate transaction_id	4 rows
Negative quantity	318 rows
Missing total_amount	247 rows
Missing category	167 rows
Missing payment_method	138 rows
Missing delivery_status	131 rows
Missing price	58 rows
Missing customer_address / name	54 / 53 rows
Invalid emails (no @ sign)	103 rows
Payment labels: creditcard, CC, credit	373 rows
Invalid / mixed date formats	e.g. 2024-02-30

Key idea: dirty data gives wrong answers. If duplicates, nulls and negatives stay in the table, every KPI built on it will be wrong.

1 Remove Duplicates

Same transaction_id appearing more than once

Approach: use **ROW_NUMBER()** inside a CTE. The first copy of each transaction_id gets 1, every repeat gets 2, 3... Then delete everything above 1.

```sql
WITH CTE AS (
   SELECT *,
      ROW_NUMBER() OVER(
         PARTITION BY transaction_id
         ORDER BY transaction_id) AS row_num
   FROM sales
)
DELETE FROM CTE
WHERE row_num > 1;
```

Metric	Value
Rows before	1,004
Duplicate rows deleted	4
Rows after	1,000

Analyst tip: always run SELECT * FROM CTE WHERE row_num > 1 first, check the rows, and only then switch to DELETE.

2 Check for NULL Values

Find missing data in every column at once

Writing **IS NULL** for 13 columns one by one is slow. I used **dynamic SQL** that reads column names from INFORMATION_SCHEMA and builds the null-count query automatically.

```sql
DECLARE @SQL NVARCHAR(MAX) = '';

SELECT @SQL = STRING_AGG(
  'SELECT ''' + COLUMN_NAME + ''' AS ColumnName,
  COUNT(*) AS NullCount FROM sales
  WHERE ' + QUOTENAME(COLUMN_NAME) + ' IS NULL',
  ' UNION ALL ')
WITHIN GROUP (ORDER BY COLUMN_NAME)
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'sales';

EXEC sp_executesql @SQL;
```

Why it matters: one query gives a full null report for the whole table. Run it again after cleaning to prove your fixes worked.

3 Treat NULL Values (Text)

Fill missing text columns + standardise labels

```sql
UPDATE sales SET category = 'Unknown'
WHERE category IS NULL;

UPDATE sales SET customer_address = 'Not Available'
WHERE customer_address IS NULL;

UPDATE sales SET payment_method = 'Credit Card'
WHERE payment_method IN ('creditcard','CC','credit');

UPDATE sales SET payment_method = 'Cash'
WHERE payment_method IS NULL;

UPDATE sales SET delivery_status = 'Not Delivered'
WHERE delivery_status IS NULL;
```

- **Standardised** 3 spellings of Credit Card into 1, so GROUP BY gives one correct total.
- **Kept the rows** and used a clear label instead of deleting them. Deleting would lose valid sales data.

> **Assumption:** I filled missing payment_method with Cash. This is a business assumption, so always write it in your documentation.

3 Treat NULL Values (Price)

Mean vs Median vs Mode, then category-wise fill

Measure	Value
Mean (overall average)	2510.76
Median	2530.75
Mode	checked with GROUP BY + COUNT

Prices differ by category, so one global value would be unfair. I filled each missing price with the **average of its own category**:

Category	Avg price
Electronics	2663.92
Books	2574.45
Clothing	2539.27
Home & Kitchen	2507.05
Toys	2235.47

```sql
UPDATE sales SET price = 2663.92
WHERE price IS NULL
  AND category = 'Electronics';
-- repeat for each category
```

> **Analyst tip:** if data has outliers, prefer the median. If it is fairly even, the mean is fine. Group-wise fill is more accurate than global fill.

4 Handle Negative Values

Fix quantity, then rebuild total_amount

```sql
-- 1) find negative quantities
SELECT * FROM sales WHERE quantity < 0;

-- 2) convert to positive
UPDATE sales SET quantity = ABS(quantity)
WHERE quantity < 0;

-- 3) recalculate total from clean columns
UPDATE sales SET total_amount = price * quantity
WHERE total_amount IS NULL
   OR total_amount <> price * quantity;

-- 4) missing customer names
UPDATE sales SET customer_name = 'User'
WHERE customer_name IS NULL;
```

- **ABS()** turned 318 negative quantities positive.
- Order matters: price and quantity are fixed **first**, then total_amount is recalculated, so it fills the 247 missing totals and corrects wrong ones.

> **Think like an analyst:** a negative quantity can mean a return, not an error. I treated it as an entry error here. In real work, confirm with the business first.

5 Fix Dates

Inconsistent formats and impossible dates

Dates were stored as text (**nvarchar**), with mixed formats and invalid values like **2024-02-30** (30th February does not exist).

```sql
UPDATE sales
SET purchase_date =
  CASE
    WHEN TRY_CONVERT(DATE, purchase_date, 103)
         IS NOT NULL
    THEN TRY_CONVERT(DATE, purchase_date, 103)
    ELSE NULL
END;
```

- **TRY_CONVERT** returns NULL for a bad date instead of throwing an error and stopping the query.
- Style **103** reads the text as dd/mm/yyyy (British format).
- Impossible dates become NULL, so they cannot spoil monthly or yearly trends.

> **Analyst tip:** CONVERT fails on bad data, TRY_CONVERT survives it. Use TRY_CONVERT while cleaning.

6 Fix Invalid Emails

Valid email must contain the @ symbol

```sql
-- find them first
SELECT * FROM sales
WHERE email NOT LIKE '%@%';

-- then fix them
UPDATE sales
SET email = NULL
WHERE email NOT LIKE '%@%';
```

Valid	Invalid (example)
caroladams@gmail.com	tammydaniels
sgibbs@reeves.org	(no @ sign)

- **102 rows** were affected.
- I set bad emails to **NULL** instead of deleting the row. The sale is still real, only the contact detail is wrong.

> **Analyst tip:** a simple LIKE '%@%' is a quick first check. For stricter validation, also test for a dot after the @ and no spaces.

7 Check & Fix Datatypes

Right datatype = faster, correct analysis

```sql
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'sales';

ALTER TABLE sales
ALTER COLUMN purchase_date DATE;
```

Column	Datatype
transaction_id, customer_id, product_id, quantity	int
price, total_amount	decimal
customer_name, email, category, payment...	nvarchar
purchase_date (before)	nvarchar
purchase_date (after)	DATE

Why it matters: as text, dates sort wrongly ("05/01" before "20/12" in any year). As DATE you can use YEAR(), MONTH(), DATEDIFF() and time-series charts.

Before vs After

The result of the 7-step cleaning

Check	Before	After
Total rows	1,004	1,000
Duplicate transaction_id	4	0
Negative quantity	318	0
NULL category / price	167 / 58	0 / 0
NULL payment / delivery	138 / 131	0 / 0
NULL total_amount	247	0
Payment label variants	7	5 clean
Invalid emails	103	0 (set NULL)
purchase_date type	nvarchar	DATE

Still NULL on purpose: customer_id (3 rows) and email (102 rows). I did not invent IDs or emails. Honest NULLs are better than fake data.

7 Golden Rules for Data Analysts

What this project taught me

1. **Keep a backup.** Never clean the only copy. Use SELECT * INTO sales_backup FROM sales.

2. **SELECT before UPDATE / DELETE.** Preview the rows. Use BEGIN TRAN and ROLLBACK to be safe.

3. **Order of steps matters.** Duplicates, then nulls, then calculations like total_amount.

4. **Choose mean, median or mode with reason.** Outliers? Use median. Categories? Use mode.

5. **Do not delete if you can fix.** Every deleted row is lost information.

6. **Document every assumption.** Cash for missing payment, ABS for negatives: write it down.

7. **Validate after cleaning.** Compare row counts, null counts, min and max values.

Skills Used & Next Steps

SQL skills practised

CTE · ROW_NUMBER() · Dynamic SQL · UPDATE / DELETE · CASE WHEN · TRY_CONVERT · PERCENTILE_CONT · ALTER TABLE · INFORMATION_SCHEMA

What I will improve next

- Review the 75 rows where quantity = 0.
- Keep text labels consistent (Unknown vs UNKNOWN, User vs user).
- Investigate the 3 rows with missing customer_id.
- Try median or mode imputation and compare results.
- Build a **Power BI dashboard** on this clean data.

Found this useful?

Like, save and repost. Comment "SQL" for the full script.